Personalized Learning Path Generator

Project Documentation

Team: Hasini, Sriya, Kushmita, Deekshitha, Uday Kiran, Haripriya

1. Executive Summary & Objectives

1.1 Problem Statement

The rapid growth of digital learning platforms has improved access to educational resources; however, most existing e-learning systems still follow a static, one-size-fits-all approach. These systems provide identical learning materials and predefined learning paths without considering differences in students' prior knowledge, learning pace, performance, and evolving requirements. This lack of personalization can result in ineffective learning outcomes, where struggling students develop conceptual gaps while advanced learners experience reduced engagement.

Recent advancements in Generative Artificial Intelligence (GenAI) have enabled automated content creation and conversational learning support, but many existing AI-based educational tools remain reactive and depend on user-initiated interactions. They lack autonomous capabilities to analyze learner progress, identify knowledge gaps, adapt study plans, and provide targeted remediation. Additionally, conventional learning systems offer limited support for continuous mastery tracking and personalized guidance at scale.

This project addresses these challenges by developing an AI-powered Personalized Learning Path Generator that combines Generative AI, Agentic AI, and Retrieval-Augmented Generation (RAG) to create an adaptive learning environment. The proposed system acts as an intelligent tutor that analyzes curriculum content, constructs knowledge representations, tracks student performance, generates personalized study materials, and autonomously adapts learning paths through planning, assessment, and feedback-driven remediation workflows.

1.2 Project Scope

In-scope

- Ingestion and parsing of syllabus/lecture documents (PDF, JSON) into a structured knowledge graph of topics and sub-topics.
- An agentic study planner that tracks student progress and autonomously decides the next best topic to study.

- An adaptive feedback loop that triggers targeted remediation content when a student underperforms on a quiz.
- Adaptive multi-modal content generation: text summaries, dynamic flashcards, varied-format quizzes, and step-by-step problem-solving guides.
- A knowledge-gap dashboard visualizing per-topic mastery levels.
- Retrieval-augmented generation (RAG) with citation-grounded answers for content questions.

Out-of-scope

- Live video tutoring or human-instructor scheduling.
- Grading of free-form written assignments beyond quiz-style assessment.
- Institution-wide LMS integration (e.g., full Canvas/Moodle sync) in the first release.
- Support for languages other than English in the initial version.

1.3 Strategic Objective

The core value proposition is a tutor that behaves like an agent, not a search box: it proactively decides what the student should study next, generates the material for it, checks understanding, and only then moves on closing the loop between content delivery and mastery, automatically and continuously. This is intended to foster deeper understanding, sustained engagement, and faster time-to-mastery than static, self-paced course material can offer.

2. System Architecture

2.1 High-Level Architecture

The system follows a layered architecture in which a browser-based frontend communicates with a backend API, which in turn invokes an AI orchestration layer responsible for all agentic decision-making. The orchestration layer calls out to an external LLM provider for generation and reasoning, and reads from and writes to a dedicated data storage layer. The diagram below shows each component and the exact path along which requests, decisions, and data travel between them.

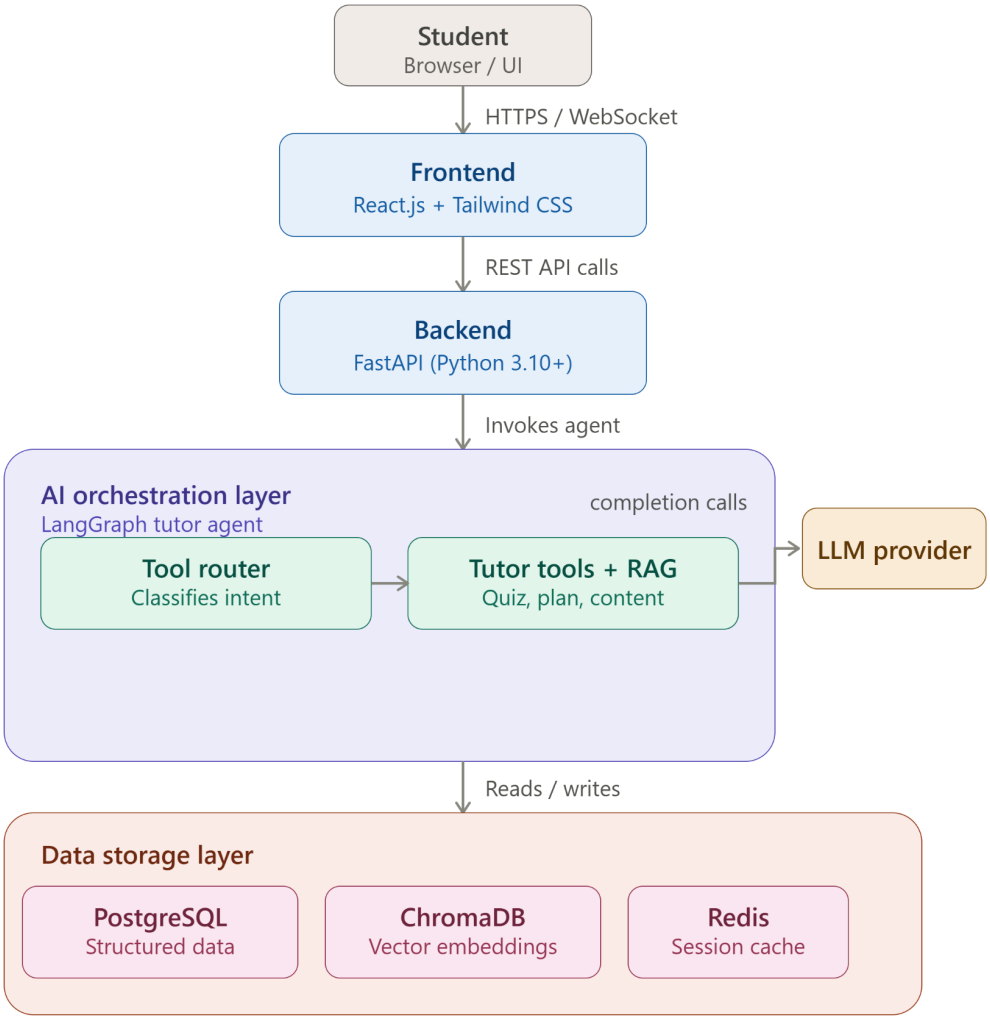


Figure 1: High-level system architecture with labeled communication paths.

Frontend layer

Purpose: Serves as the interactive interface for students: syllabus uploads, chat-based tutoring, quiz participation, and progress visualization.

Key technologies: React.js / Next.js, Tailwind CSS.

Communication paths: Student's browser communicates with the frontend over standard web protocols (HTTPS for page loads and REST calls, WebSocket for the live chat/tutor session). The frontend, in turn, issues REST requests to the backend (e.g., /syllabus/upload, /study-plan/generate) and opens a WebSocket connection for streaming chat responses.

Backend layer

Purpose: Exposes the application's API surface, handles authentication and session state, and forwards requests into the AI orchestration layer.

Key technologies: Python 3.10+, FastAPI (asynchronous request handling).

Communication paths: Receives REST/WebSocket calls from the frontend, invokes the LangGraph tutor agent with the student's request and session context, and returns the agent's response (content, citations, status) back to the frontend.

AI orchestration layer

Purpose: Owns all agentic decision-making: interprets the student's intent, decides which tool should handle the request, and composes the final response.

Key technologies: LangChain / LangGraph.

Internally, this layer contains a Tool Router (an LLM-driven classifier) and a set of Tutor Tools (quiz generation, evaluation, knowledge-gap analysis, remediation, study planning, and multi-modal content generation). On receiving a request from the backend, the Tool Router first classifies intent. If the intent is a content question, the request is routed to the RAG/citation engine; if the intent is an action (generate quiz, evaluate, plan study, generate study kit), it is routed to the relevant Tutor Tool. Both paths call out to the external LLM provider for generation, and both read from and write to the data storage layer ChromaDB for retrieval, PostgreSQL for state and history, and Redis for session memory.

External LLM provider

Purpose: Performs the underlying language generation and reasoning that powers every tool in the orchestration layer.

Key technologies: OpenAI GPT-4o (or GPT-4o-mini for cost-sensitive calls), Llama 3 via Ollama for local development.

Communication paths: The orchestration layer sends completion requests to the LLM provider and receives generated text, which is then post-processed (citation attachment, formatting) before being returned to the backend.

Data storage layer

Purpose: Persists all structured, vector, and session data the system depends on.

Key technologies: PostgreSQL (structured data — user profiles, knowledge graph, quiz history, mastery scores), ChromaDB (vector embeddings for semantic retrieval), Redis (session memory and short-lived cache).

Communication paths: The orchestration layer queries ChromaDB for semantically relevant chunks during retrieval, reads/writes PostgreSQL for durable state (progress, scores, the knowledge graph), and reads/writes Redis for fast-access session context.

2.2 Data Ingestion Pipeline

- Upload: student or instructor uploads a syllabus or lecture document (PDF/JSON) via the frontend.
- Parsing: the backend extracts raw text and structural metadata (headings, page boundaries) from the document.

- Chunking: parsed text is split into semantically coherent chunks sized for retrieval.
- Topic extraction: an LLM call processes the chunks to construct a hierarchical Knowledge Graph of topics and sub-topics, enriched with estimated difficulty levels. This graph is stored in PostgreSQL and underpins adaptive planning and knowledge-gap analysis.
- Vectorization: each chunk is embedded and stored in ChromaDB for semantic search.
- Refresh: re-uploaded or updated documents trigger re-ingestion so the knowledge graph and vector store stay current.

2.3 Component Interaction

A typical request travels from the student's browser to the frontend, over REST or WebSocket to the backend, and into the LangGraph tutor agent. The Tool Router classifies the request and directs it either to the RAG/citation engine (for content questions, which queries ChromaDB and synthesizes a cited answer) or to a Tutor Tool (for actions such as generating a quiz or updating the study plan, which reads/writes PostgreSQL and Redis as needed). In both cases, the relevant tool calls the external LLM provider to generate the final content, and the composed response including any citations is returned back up through the backend to the frontend for display.

3. Functional Requirements

3.1 User Stories

Role	User story
Student	As a student, I want the tutor to generate a study plan from my syllabus, so that I know what to study and in what order.
Student	As a student, I want to take short quizzes on topics I've studied, so that the system can measure my mastery.
Student	As a student, I want the tutor to proactively check in on topics I haven't mastered, so that I don't forget material I've already covered.
Student	As a student, I want to upload my lecture notes and get a structured Study Kit (summary, flashcards, quiz), so that I can review efficiently.
Student	As a student, I want a step-by-step problem-solving guide for a complex topic, so that I understand the logical progression rather than just the final answer.
Student	As a student, after failing a quiz, I want a micro-learning module targeting the specific sub-concept I struggled with, so that I can remediate without re-studying the entire topic.
Student	As a student, I want the tutor to explain why my incorrect quiz answers were wrong, so that I can learn from the mistake rather than just seeing a score.
Instructor	As an instructor, I want a dashboard of per-topic mastery across my class, so that I can see where students are struggling collectively.

3.2 Acceptance Criteria

- The system must return a citation-grounded answer to a content question within 3 seconds (P95).
- Generation of a complete Study Kit (summary, 5 flashcards, 5-question quiz, and one step-by-step guide) from uploaded notes must complete within 20 seconds (P95).
- A micro-learning remediation module must be available within 5 seconds (P95) of quiz submission.

- Proactive check-in suggestions must be generated within 1 second (P95) of a triggering event (quiz submission, session completion).
- The system must support at least 50 concurrent student sessions without a degradation in the above latency targets.

4. Technical Specifications

4.1 Technology Stack

Layer	Technology	Why
Frontend	React.js / Next.js + Tailwind CSS	Responsive, component-driven UI with fast iteration on chat and dashboard views.
Backend	Python 3.10+, FastAPI	Asynchronous request handling suited to concurrent, latency-sensitive AI calls.
Agentic framework	LangChain / LangGraph	Native support for tool routing, stateful agent loops, and multi-step reasoning.
LLM	OpenAI GPT-4o (prod), Llama 3 via Ollama (local dev)	GPT-4o for production quality; a local open-weight model removes API cost/latency during development.
Vector store	ChromaDB	Lightweight, easy-to-embed semantic search for RAG retrieval.
Relational store	PostgreSQL 15	Durable storage for the knowledge graph, quiz history, and mastery scores.
Cache / session store	Redis	Low-latency session memory for the agent's short-term context.

4.2 API Documentation

Endpoint	Method	Description
/syllabus/upload	POST	Upload a syllabus/lecture document for ingestion into the knowledge graph.
/study-plan/generate	POST	Generate or refresh the student's adaptive study plan.
/chat	WS	Streaming chat channel between student and tutor agent.
/study-kit/generate-advanced	POST	Generate a multi-modal study kit (summary, flashcards, quiz, step-by-step guide).
/remediation/micro-module	POST	Trigger a micro-learning module for a specific sub-concept after a failed quiz.
/quiz/explain-answer	POST	Return a detailed explanation for an incorrect quiz answer.
/progress/trends	GET	Retrieve performance trend data used for proactive check-ins.

Authentication: session-based auth using a signed token issued at login and passed on every request (REST) or during the WebSocket handshake.

4.3 Infrastructure Requirements

- Deployment: Dockerized services (frontend, backend, orchestration layer) orchestrated locally via Docker Compose, with a path to a single-node cloud deployment (AWS EC2 or equivalent) for staging/demo.
- Resource constraints: backend + orchestration layer target 2 vCPU / 4GB RAM per instance for the expected concurrent-session load; PostgreSQL and Redis run as managed or containerized services with persistent volumes.

5. AI & Agentic Framework Logic

5.1 Prompt Engineering Strategy

The system prompt fixes the agent's role as a tutor rather than a general assistant, instructing it to prefer invoking a tool over answering from parametric memory whenever a tool exists for the task. A specialized prompt template is used for the multi-modal content generator to keep generated formats (summary, flashcard, quiz, step-by-step guide) structurally consistent, and few-shot examples are used for step-by-step guide generation to keep the reasoning steps in a predictable, gradable structure.

5.2 RAG Implementation

- Chunking strategy: semantically coherent chunks (rather than fixed-length windows) sized for retrieval quality.
- Retrieval: hybrid approach combining semantic search over ChromaDB with a keyword fallback for exact-match queries (e.g., specific terms, formulas).
- Synthesis: retrieved chunks are passed to the citation-aware synthesis step, which grounds every generated answer in the retrieved source and attaches citations.

5.3 Agentic Decision-Making the Agent Loop

On each incoming request, the Tool Router first classifies intent. A content question is routed to the RAG/citation engine. An action request is routed to the matching Tutor Tool: Study Planner (schedule/re-schedule), Quiz Generator, Quiz Evaluator, Knowledge Gap Analyzer, Remediation Tool, or Multi-Modal Content Generator. After the Quiz Evaluator scores an attempt, a score below the mastery threshold triggers the Remediation Tool, which generates a micro-learning module and a detailed explanation of the incorrect answers before allowing the student to advance. Independently, performance trends read from PostgreSQL/Redis feed the Study Planner, which can proactively surface a check-in suggestion rather than waiting for the student to ask.

5.4 Guardrails & Hallucination Control

- Evidence-bound answers: the CitationQueryEngine is instructed to answer only from retrieved chunks and to decline when similarity confidence is low, rather than filling gaps from the base LLM's parametric knowledge.
- Citation enforcement: every content answer must carry at least one source citation before it is returned to the student; responses without one are treated as a synthesis failure and retried once.

- Score gating: The Remediation Workflow is a hard gate, a student cannot be advanced to the next topic by the agent while their mastery score for the current topic is below the configured threshold.
- Latency guardrail: retrieval is capped at a fixed top-k and a query timeout; if the vector search or LLM call exceeds budget, the agent returns a fast partial answer rather than blocking indefinitely, protecting the 3-second target under load.

6. Security & Compliance Standards

- **6.1 Data Privacy**
  - Uploaded documents and chat transcripts are scoped per user account; no cross-student data is retrievable by another student's session.
  - Any PII detected in uploaded notes (names, emails, phone numbers) is scrubbed before the content is embedded into ChromaDB, so raw PII never enters the vector store.
  - Passwords are hashed (bcrypt/argon2); no plaintext credentials are stored or logged.
- **6.2 Access Control**
  - Authentication via JWT issued at login; tokens are short-lived with refresh-token rotation.
  - Authorization is role-based (student vs. instructor); instructor-only endpoints (cohort dashboard) enforce role checks at the FastAPI dependency layer.
  - All API traffic is served over HTTPS; secrets (DB credentials, LLM API keys) are read from environment variables, never committed to source control.
- **6.3 Compliance**
- The system is designed to align with standard data-protection practice for handling student records: data minimization (only storing what the tutor needs to function), scoped access, and the PII-scrubbing step above. Any institution-specific compliance regime (e.g., FERPA-equivalent policy) can be layered on top by restricting data residency and retention at the deployment stage. Handling of student educational records follows standard data-minimization and access-logging practices appropriate for an academic deployment.

7. Success Metrics & Validation (KPIs)

7.1 Performance Benchmarks

- Latency: P95 response time per endpoint as specified in acceptance criteria (Section 3.2).
- Throughput and availability: sustained concurrent-session target with uptime tracked during demo/staging.

7.2 Accuracy Metrics

- Precision, recall, and F1-score for retrieval relevance (RAG chunk retrieval) and for knowledge-gap identification against expert-labeled samples.
- Human-rated quality (1–5 scale) for generated step-by-step guides and study kits.

7.3 User Adoption

- Expected usage frequency of the study planner and study-kit generation features.
- Reduction in time-to-mastery for remediated topics compared to a non-adaptive baseline.